信用全景查询服务（xyqj）· API 接口文档与使用手册

> 面向接入方（客户）技术与运维人员的对外接口说明。
> 版本：xyqj（信用全景）｜通信：HTTP + JSON｜编码：UTF-8
> 说明：本服务采用统一的请求信封与 MD5 加签方式，响应分为 `head`（网关头部）与 `body`（业务结果）两部分。其余内部处理细节调用方无需关心。
> 关键特性：信用全景的查询结果是一个结构化对象（信用评分、借贷履约指标与多维度交易统计），本服务将其原样序列化为 JSON 字符串置于 `body.result.range`，由调用方自行解析（见 3.1.5 result.range 结构说明）。
> 计费提示：仅查得数据（`001`）计费，查无结果（`999`）不计费，详见「五、计费说明」。

一、接入必读

1.1 适用范围

本文档适用于接入本平台「信用全景查询服务（xyqj）」的第三方产品技术开发人员与日常运维人员。

1.2 接入须知

1. 正式服务地址（任选其一，两域名等价）：`http://aiszcloud.cn:8080`、`http://aiszcloud.com.cn:8080`。
2. 接入前需先申请开通账户，由我方分配 `appKey` 与 `appSecret`（加签密钥）。
3. 接入时需向我方提供贵司的外网出口 IP，以便加入访问白名单。

1.3 接口说明

项目	说明
请求方式	POST（成功查得数查询为 GET）
通信协议	HTTP
数据格式	请求体与响应体均为 JSON
字符编码	UTF-8
超时时间	4 秒（建议客户端读超时 ⩾ 5 秒）
HTTP 状态码	恒为 200；业务结果与错误均通过响应体的 `head.errorCode` / `body.code` 表达
签名	调用时需对 `body` 中的业务参数 + 我方分配的 `appSecret` 进行 MD5 加签，详见「二、鉴权与加签」

1.4 环境说明

- 正式环境：使用正式账户，调用已开通接口，按查得成功条数计费（见「五、计费说明」）。
- 测试环境：使用测试账户，返回挡板/联调数据。
- 正式账户仅适用于正式环境，测试账户仅适用于测试环境。

二、鉴权与加签

所有业务接口共用同一套请求信封与鉴权方式。

2.1 请求信封（顶层参数）

参数	示例	类型	必填	说明
appKey	y890xyqj	String	是	我方分配给客户的公开标识
sign	0528999dd55c025b8f36fc72dceb1f63	String	是	对 body 业务参数的 MD5 签名（见 2.3）
encryptionType	1	int	否	参数加密类型，1 = 明文（默认）
body	{...}	Object	是	业务请求体，见各接口定义

注意：本服务不需要 `apiKey` 参数（产品由请求路径区分）。`appKey` / `sign` / `encryptionType` 不参与签名计算。调用方传入的 `body` 字段一律为明文。

2.2 鉴权校验顺序

网关按以下顺序校验，任一失败立即返回对应 `head.errorCode`（直接拒绝，不计费）：

1. `appKey` 是否存在 → 否则 `505001`
2. `appKey` 是否匹配到账户 → 否则 `505004`
3. 账户是否有效（启用且在有效期内）→ 否则 `505007`
4. 签名是否正确 → 否则 `505002`

2.3 加签方式

1. 取出 `body` 中所有非空的业务参数（不含文件/字节流类型，不含值为空的参数）。
2. 按参数名（key）的 ASCII 升序排序；首字符相同则依次比较后续字符。
3. 将排序后的参数按 `参数名参数值` 直接拼接，最后追加 `appSecret`，得到待签名串。
4. 对待签名串做 MD5，取 32 位小写十六进制字符串，赋值给 `sign`。

示例：`body = { "name": "张三", "idCard": "360429199806073112", "mobile": "13209101123" }`，`appSecret = "<你的密钥>"`。

按 ASCII 升序排序后 key 顺序为 `idCard → mobile → name`，待签名串为：

```
idCard360429199806073112mobile13209101123name张三<你的密钥>
```

`sign = MD5(待签名串)` 的小写十六进制值。

提示：拼接顺序由 key 的 ASCII 决定，请勿写死字段顺序；新增字段时排序会自动变化。

2.4 加签代码示例

Java

```
public static String sign(Map<String, String> bodyParams, String appSecret) throws
    StringBuilder sb = new StringBuilder();
    List<String> keys = new ArrayList<>(bodyParams.keySet());
    Collections.sort(keys); // ASCII 升序
    for (String k : keys) {
        String v = bodyParams.get(k);
        if (v == null || v.isEmpty()) continue; // 剔除空值
        sb.append(k).append(v);
    }
    sb.append(appSecret);
    MessageDigest md = MessageDigest.getInstance("MD5");
    byte[] digest = md.digest(sb.toString().getBytes(StandardCharsets.UTF_8));
    StringBuilder hex = new StringBuilder();
    for (byte b : digest) hex.append(String.format("%02x", b));
    return hex.toString();
}
```

Python

```
import hashlib

def sign(body_params: dict, app_secret: str) -> str:
    parts = []
    for k in sorted(body_params.keys()):   # ASCII 升序
        v = body_params[k]
        if v is None or v == "":
            continue                        # 剔除空值
        parts.append(f"{k}{v}")
    raw = "".join(parts) + app_secret
    return hashlib.md5(raw.encode("utf-8")).hexdigest()  # 小写 hex
```

Go

```
func sign(body map[string]string, appSecret string) string {
    keys := make([]string, 0, len(body))
    for k, v := range body {
        if v != "" { // 剔除空值
            keys = append(keys, k)
        }
    }
    sort.Strings(keys) // ASCII 升序
    var sb strings.Builder
    for _, k := range keys {
        sb.WriteString(k)
        sb.WriteString(body[k])
    }
    sb.WriteString(appSecret)
    sum := md5.Sum([]byte(sb.String()))
    return hex.EncodeToString(sum[:]) // 小写 hex
}
```

三、接口列表

3.1 信用全景查询（xyqj）

项目	内容
路径	`POST /v1/openapi/zlx/querySrmxXYQJ`
完整地址	`http://aiszcloud.cn:8080/v1/openapi/zlx/querySrmxXYQJ`（或 `http://aiszcloud.com.cn:8080/v1/openapi/zlx/querySrmxXYQJ`）
鉴权	appKey + MD5 签名（见第二章）

3.1.1 请求 body 参数

参数	示例	类型	必填	说明
name	张三	String	是	姓名，明文传入，不能为空
idCard	360429199806073112	String	是	身份证号（18 位，末位可为 `X`），明文传入
mobile	13209101123	String	是	手机号（11 位，以 1 开头），明文传入

本接口 `name`、`idCard`、`mobile` 三项**均必填**；额外传入的字段不参与校验、亦不影响结果。参数缺失或格式非法（姓名为空、身份证号或手机号不符）将返回 `head.errorCode = 505062`，直接拒绝，不计费。

3.1.2 请求示例

```
{
  "encryptionType": 1,
  "appKey": "y890xyqj",
  "sign": "0528999dd55c025b8f36fc72dceb1f63",
  "body": {
    "name": "张三",
    "idCard": "360429199806073112",
    "mobile": "13209101123"
  }
}
```

3.1.3 响应结构

响应分为 `head`（网关头部）与 `body`（业务结果）两部分：

head 字段：

参数	示例	类型	说明
errorCode	0	String	网关返回码。`0` = 成功（含查得/查无）；非 0 = 网关级错误，此时无 body
errorMsg	success	String	返回描述
logId	a1b2c3...	String	全链路追踪 ID，排障/对账时请提供
time	128	Number	服务处理耗时（毫秒）
timestamp	1718456789012	Number	响应时间戳（毫秒）

body 字段（仅 `head.errorCode = 0` 时返回）：

参数	示例	类型	说明
code	001	String	业务结果码。`001` = 查得数据【计费】；`999` = 查无结果【不计费】
msg	成功	String	业务描述
reqid	lkf9x2...	String	本次请求流水号
uid	lkf9x2...	String	交易流水号（对账用）
result	{...}	Object	业务内容，仅 `code = 001` 时存在
result.range	"{...}"	String	信用全景结果对象的 JSON 字符串，调用方需 `JSON.parse` 后使用，结构见 3.1.5

3.1.4 响应示例

① 查得数据（计费）

```
{
  "head": { "errorCode": "0", "errorMsg": "success", "logId": "a1b2c3d4", "time": 1
  "body": {
    "code": "001",
    "msg": "成功",
    "reqid": "lkf9x2ab",
    "uid": "lkf9x2ab",
    "result": {
      "range": "{\"xyp_model_score_high\":\"668\",\"xyp_model_score_mid\":\"765\",\
    }
  }
}
```

② 查无结果（不计费）

```
{
  "head": { "errorCode": "0", "errorMsg": "success", "logId": "a1b2c3d6", "time": 9
  "body": {
    "code": "999",
    "msg": "查无结果",
    "reqid": "lkf9x2ad",
    "uid": "lkf9x2ad"
  }
}
```

`999` 表示本次查询未产出可计费的结果（如库中无该主体的可用记录），不计费（见「五、计费说明」）。

③ 网关级错误（无 body）

```
{
  "head": { "errorCode": "505002", "errorMsg": "账号信息异常", "logId": "a1b2c3d7", "
}
```

3.1.5 result.range 结构说明

`result.range` 是信用全景结果对象序列化后的 JSON 字符串（完整结果原样返回，未做裁剪）。调用方应先对该字符串做一次 `JSON.parse` / 反序列化，再按字段名读取。**所有字段值均为字符串**；值为空字符串 `""` 表示该指标无数据。除注明为评分、指数或比值的字段外，计数/金额/天数类指标均为**区间化编码值**（编码与区间的对照表可向我方商务索取）。

评分与综合指数：

字段	含义
`xyp_model_score_high`	小额网贷评分，范围 [350, 950]，越大逾期率越低；未命中为 `-1`
`xyp_model_score_mid`	小额分期评分，范围 [350, 950]，越大逾期率越低；未命中为 `-1`
`xyp_model_score_low`	中大额分期评分，范围 [350, 950]，越大逾期率越低；未命中为 `-1`
`xyp_cpl0081`	信用风险评分，取值 0—1，分数越高用户信用越低
`xyp_cpl0082`	履约金额综合指数，取值 0—1，指数越大用户逾期可能性越高
`xyp_cpl0083`	履约笔数综合指数，取值 0—1，指数越大用户逾期可能性越高

借贷履约指标（`xyp_cpl` 前缀，字段名为 `xyp_cpl` + 四位编号，如 `xyp_cpl0001`）：

编号	含义
0001	贷款总机构数
0002	贷款已结清机构数
0007	消费金融类机构数（有场景的、分期）
0008	网络贷款类机构数（现金贷）
0070	最近 1 天贷款机构数
0009	最近 7 天贷款机构数
0010	最近 14 天贷款机构数
0063	最近 21 天贷款机构数
0011	最近 30 天贷款机构数
0012	最近 90 天贷款机构数
0013	最近 180 天贷款机构数
0014	历史贷款机构成功还款笔数
0015	历史贷款机构交易失败笔数
0016 / 0017	最近 1 天交易失败笔数 / 还款成功笔数
0018 / 0019	最近 7 天交易失败笔数 / 还款成功笔数
0020 / 0021	最近 14 天交易失败笔数 / 还款成功笔数
0065 / 0064	最近 21 天交易失败笔数 / 还款成功笔数
0022 / 0023	最近 30 天交易失败笔数 / 还款成功笔数
0024 / 0025	最近 90 天交易失败笔数 / 还款成功笔数
0026 / 0027	最近 180 天交易失败笔数 / 还款成功笔数
0028 / 0029 / 0030 / 0031	最近 1 / 7 / 14 / 30 天是否发生过逾期
0032 / 0033	最近 1 天交易失败总金额 / 还款成功总金额
0034 / 0035	最近 7 天交易失败总金额 / 还款成功总金额
0036 / 0037	最近 14 天交易失败总金额 / 还款成功总金额
0066 / 0067	最近 21 天交易失败总金额 / 还款成功总金额
0038 / 0039	最近 30 天交易失败总金额 / 还款成功总金额
0040 / 0041	最近 90 天交易失败总金额 / 还款成功总金额

编号	含义
0042 / 0043	最近 180 天交易失败总金额 / 还款成功总金额
0044	当前是否存在逾期未结清
0045	信用贷款时长
0046	最近一次交易距离当前时间
0048 / 0049 / 0050 / 0051	最近 7 / 14 / 21 / 30 天交易失败次数 > 0 的机构数
0052 / 0053 / 0069	消费金融类 / 小贷担保类 / 全部 最后一次交易失败后还款次数
0054 / 0055 / 0056	消费金融类 / 小贷担保类 / 全部 交易失败向后距离下一次还款成功的天数最大值
0057 / 0058 / 0059	消费金融类 / 小贷担保类 / 全部 交易失败向后距离下一次还款成功的天数最小值
0060 / 0061 / 0062	消费金融类 / 小贷担保类 / 全部 交易失败向后距离下一次还款成功的天数平均值
0068	最近一次还款成功距离当前天数
0071	当前逾期机构数
0072	当前逾期金额
0073	近 5 次还款中成功还款总金额 / 近 5 次还款总金额（比值）
0074	近 5 次还款中还款成功笔数 / 近 5 次还款总笔数（比值）
0075	近 20 次还款中小贷担保类还款成功笔数 / 近 20 次还款中小贷担保类还款笔数（比值）
0076	近 20 次还款中还款成功笔数
0077	近 5 次还款中还款成功总金额
0078	近 20 次还款中还款成功总金额
0079	近 90 天内还款成功总金额 / 近 90 天还款总金额（比值）
0080	近 90 天内还款成功笔数 / 近 90 天内还款总笔数（比值）

统计类指标（`xyp_t01` 前缀，单项统计）命名规则：`xyp_t01` + 统计方式（2 位）+ 时间窗口（1 位）+ 新增机构窗口（1 位）+ 机构类别（1 位）+ 交易状态（1 位）。

位置	代码 → 含义
统计方式	`aa` 交易金额最大值；`ab` 交易金额最小值；`ac` 交易金额求和；`ad` 交易金额平均值；`ba` 交易距离当前天数最大值；`bb` 交易距离当前天数最小值；`cc` 笔数求和；`de` 机构数去重计数
时间窗口	`b` 近 7 天；`c` 近 14 天；`d` 近 21 天；`e` 近 30 天；`f` 近 90 天；`g` 近 180 天；`h` 近 360 天；`i` 近 5 次交易；`j` 近 20 次交易；`k` 近 50 次交易；`l` 近 100 次交易；`z` 不限
新增机构窗口	`e` / `f` / `g` / `h` 最近 30 / 90 / 180 / 360 天新增机构；`z` 不限
机构类别	`a` 消费金融类；`b` 小贷担保类；`z` 全部
交易状态	`a` 因交易能力不足导致失败；`c` 还款成功（金额类指标取还款金额）；`z` 全部交易

举例：`xyp_t01aafzbc` = 近 90 天_小贷担保类_还款成功_还款金额_最大值；`xyp_t01aazfbc` = 最近 90 天新增机构_小贷担保类_还款成功_还款金额_最大值；`xyp_t01cchzaz` = 近 360 天_消费金融类_笔数_求和；`xyp_t01dekzzz` = 近 50 次交易_机构数_去重计数。

其他指标：

字段	含义
`xyp_t02<分子>_<分母>`	两项统计指标之比（比值，非区间编码），分子与分母按上述 `t01` 规则解读，如 `xyp_t02achzzc_achzzz` = 近 360 天_还款成功_还款金额_求和 / 近 360 天_交易金额_求和
`xyp_t03td045` / `xyp_t03td053` / `xyp_t03td061`	最近 90 / 180 / 360 天_小贷担保类_交易失败次数 > 0 的机构数
`xyp_t03td111` / `xyp_t03td115`	最近 90 / 180 天_交易失败次数 > 0 的机构数
`xyp_t03td148`	最近一次交易为交易失败的机构数
`xyp_t0400002` / `xyp_t0400003` / `xyp_t0400004`	近 20 / 50 / 100 次还款_还款成功笔数 / 还款笔数（比值）
`xyp_var1` ~ `xyp_var10`	预留字段
`fee_status`	计费状态标识；`001` 响应中如出现，恒为 `"1"`（本次计费）

说明：`result.range` 的具体字段可能随数据能力升级扩展，本服务保证完整透出。建议调用方按「存在即解析、缺失即忽略」的方式做容错处理，避免因后续新增字段导致解析失败。

解析示例（JavaScript）：

```
const range = JSON.parse(resp.body.result.range);
const num = v => (v === undefined || v === "" ? null : Number(v)); // "" 表示无数据
const scoreHigh = num(range.xyp_model_score_high); // -1 表示未命中
const creditRisk = num(range.xyp_cpl0081);          // 0-1，越高信用越低
if (scoreHigh !== null && scoreHigh !== -1) {
  console.log("小额网贷评分", scoreHigh, "信用风险评分", creditRisk);
}
```

3.2 成功查得数查询（扩展接口）

查询本账户累计成功查得数与调用次数，用于客户侧自助监控与对账。不返回额度上限/剩余量（本服务无额度限制）。

项目	内容
路径	`GET /v1/openapi/zlx/quotaXYQJ`
完整地址	`http://aiszcloud.cn:8080/v1/openapi/zlx/quotaXYQJ`（或 `http://aiszcloud.com.cn:8080/v1/openapi/zlx/quotaXYQJ`）
鉴权	与主接口一致（请求体中携带 `appKey` + `sign` 信封；`body` 可为 `{}`，此时 `sign = MD5(appSecret)`）

响应示例

```
{
  "errorCode": "0",
  "errorMsg": "success",
  "status": "ACTIVE",
  "serviceUsed": 1280,
  "totalCalls": 1530
}
```

参数	说明
status	账户状态（ACTIVE/SUSPENDED 等）
serviceUsed	累计成功查得数据的次数（仅统计查得成功 `code = 001`）
totalCalls	累计调用次数（含查得、查无及已受理的请求）

说明：无任何额度上限拦截，仅做成功查得数与调用次数统计。

3.3 健康检查

项目	内容
路径	`GET /healthz`
完整地址	`http://aiszcloud.cn:8080/healthz`（或 `http://aiszcloud.com.cn:8080/healthz`）
鉴权	无
响应	HTTP 200，纯文本 `ok`

四、返回码说明

4.1 网关返回码 head.errorCode

errorCode	含义	典型原因
0	成功	调用成功（业务结果见 `body.code`）
505001	appKey 异常	缺少或非法 appKey
505004	账户信息不存在	appKey 未匹配到账户
505007	服务尚未开通	账户停用 / 过期 / 未开通
505002	账号信息异常	签名校验失败
505003	产品编号异常	保留
505062	数据请求异常	参数非法 / 服务处理异常 / 超时未决 / 系统错误（默认错误码）

服务处理异常（账户/参数/系统/超时未决等）统一返回 `505062`，不计费，并经异步复查兜底。

4.2 业务结果码 body.code（仅 errorCode = 0 时）

code	含义	是否计费
001	查得数据	计费
999	查无结果	不计费

五、计费说明

- 仅当返回 `body.code = 001`（查得数据）时，才计入成功查得数并对客户计费。
- `body.code = 999`（查无结果）**不计费**。
- 网关级错误（`head.errorCode` 非 0：鉴权失败、参数非法、服务处理异常、系统异常等）一律不计费。

- 计费以最终落库的台账为准，超时未决请求会经异步复查/对账裁定状态，不会重复计费。
- 成功查得数可随时通过「成功查得数查询（3.2）」接口自助核对。

六、使用手册（接入与最佳实践）

6.1 接入流程

1. 向商务申请账户，获取 `appKey`、`appSecret`，并提供贵司外网出口 IP 用于加白；服务地址见「1.2 接入须知」。
2. 按第二章实现加签，先在测试环境联调，再切正式环境。
3. 上线后通过「成功查得数查询（3.2）」接口监控调用量与计费次数。

6.2 幂等与重试

- 客户端建议为每笔查询设置合理超时（≥ 5 秒）。
- 收到网络超时/无响应时可安全重试：网关基于内部流水号做幂等，不会因重试重复计费。
- 请勿对已明确返回 `head.errorCode` 的请求做无差别重试（如参数错误 `505062`、鉴权错误 `505001` / `505002` / `505004`），应先修正再发起。

6.3 错误处理建议

现象	排查方向
505001 / 505004	检查 appKey 是否正确、是否用错环境账户
505002	检查签名算法（排序/空值剔除/UTF-8/小写 hex）
505007	联系商务确认账户状态与有效期
505062	检查 name / idCard / mobile 是否齐全合法；若入参正常仍持续出现，记录 logId 联系我方

任何异常排查请一并提供响应中的 `head.logId`，便于我方全链路定位。

6.4 联调自检清单

- [ ] 服务地址（aiszcloud.cn:8080 或 aiszcloud.com.cn:8080）、appKey、appSecret、环境匹配无误，出口 IP 已报我方加白
- [ ] 待签名串严格按 ASCII 升序拼接、剔除空值、UTF-8、MD5 小写
- [ ] name / idCard / mobile 三项均以明文传入且齐全合法
- [ ] 能正确解析 `head.errorCode` 与 `body.code` 两级状态
- [ ] 已知悉仅 `001` 查得计费、`999` 查无不计费
- [ ] 已实现对 `result.range` 字符串的二次 `JSON.parse` 解析与字段容错
- [ ] 已实现超时重试（依赖幂等，不重复计费）

附录：术语表

术语	说明
appKey	公开账户标识，随请求明文传输
appSecret	加签密钥，仅本地保存用于计算 sign，切勿泄露或随请求传输
logId	全链路追踪 ID（= `head.logId`），排障/对账唯一凭据
range	信用全景结果的 JSON 字符串，需二次解析（结构见 3.1.5）
区间化编码值	计数/金额/天数类指标按区间分档后的编码，而非原始数值